Intelligent Data Pipeline Scheduler

An AI Agent for Adaptive Pipeline Scheduling Using Search,
Knowledge Representation, Constraint Satisfaction, and Learning

Graduate AI — Final Project Report | Option D: Planning & Scheduling

Sanjana • Raghava Thyagaraj • Monisha • Diogo Dcosta • Rakshitha

Pace University — CS 627 Artificial Intelligence — May 2026

Abstract

Static pipeline schedulers operate on fixed timetables and cannot adapt to failures, runtime variance, or resource contention. This project presents an intelligent scheduling agent integrating four AI techniques: a first-order logic knowledge base with forward and backward chaining for proactive risk detection, a CSP formulation with backtracking for valid schedule generation, A* search for failure recovery, and EWMA learning for improving runtime estimates. Evaluated across three pipeline configurations (5, 10, 20 tasks), the CSP solver achieves zero constraint violations with 100% SLA adherence, the learning component reduces estimation error by 30.2% over ten iterations, and the A* planner successfully recovers from task failures. The system comprises ~3,700 lines of Python validated by 176 automated tests.

1. Introduction and Problem Formulation

1.1 Problem Description and Significance

Data-driven organizations rely on nightly batch pipelines — directed acyclic graphs (DAGs) of tasks with dependencies, resource requirements, and SLA deadlines. Current schedulers (cron, Airflow) follow static timetables and cannot reason about constraints, adapt to failures, or learn from history. When tasks fail or take longer than expected, downstream work cascades into failure, SLAs are missed, and engineers are paged. This project builds an intelligent agent that reasons, plans, recovers, and learns.

1.2 PEAS Specification

Component	Specification
Performance	SLA adherence (target 100%), makespan minimization, resource utilization, recovery rate
Environment	Partially observable, stochastic, sequential, dynamic, discrete, multi-agent (shared resources)
Actuators	Schedule task, retry failed task, skip task, reallocate resource, wait
Sensors	Task status monitor, resource capacity monitor, clock, failure detector

Table 1: PEAS specification.

1.3 Environment Characterization

The environment is **partially observable** (future runtimes unknown), **stochastic** (runtimes vary, tasks may fail), **sequential** (each decision constrains future options), **dynamic** (state changes during execution), **discrete** (finite time slots, integer CPUs), and involves **multiple competing tasks** on shared resources.

1.4 Agent Type and Success Criteria

The agent is a **learning utility-based agent**: utility-based because it evaluates and optimizes schedule quality (SLA adherence × makespan × utilization), and learning because EWMA estimation improves decisions across iterations. Success criteria: (1) zero constraint violations, (2) A* finds recovery plans, (3) learning reduces estimation error, (4) agent matches or beats the naive baseline.

2. System Architecture

2.1 Overall Design

The system has five layers: Data (task_dag.py — DAG model), Knowledge (knowledge_base.py — FOL inference), Constraint (csp_solver.py — CSP scheduling), Planning (search_planner.py — A* recovery), and Learning (learning.py — EWMA). The Scheduler Agent (scheduler_agent.py) orchestrates all components. Figure 1 shows the architecture; Figure 2 shows component interactions.

Figure 1: System Architecture — Five-Layer Design

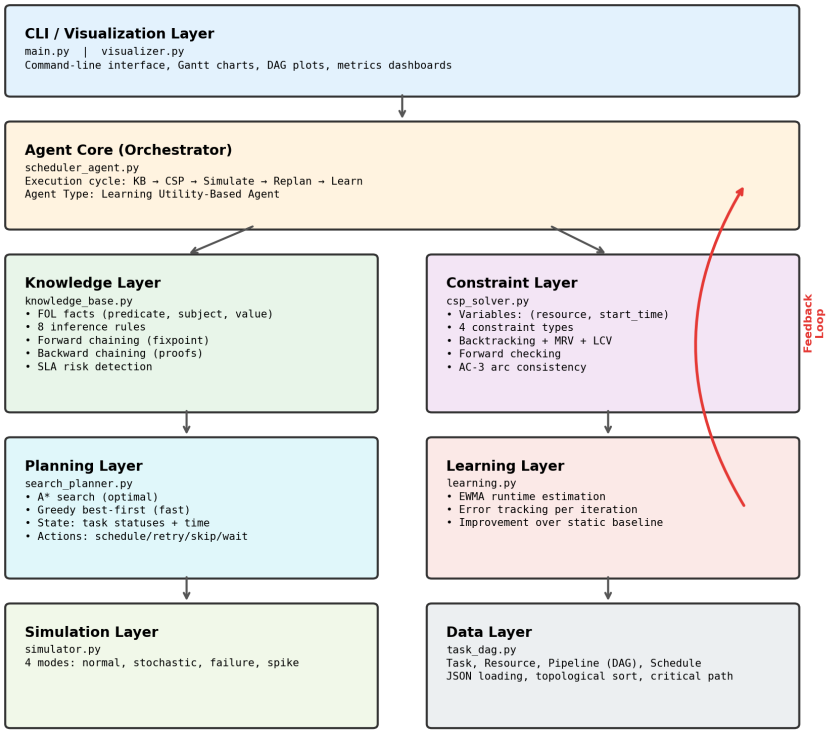

Figure 1: Five-layer system architecture with feedback loop.

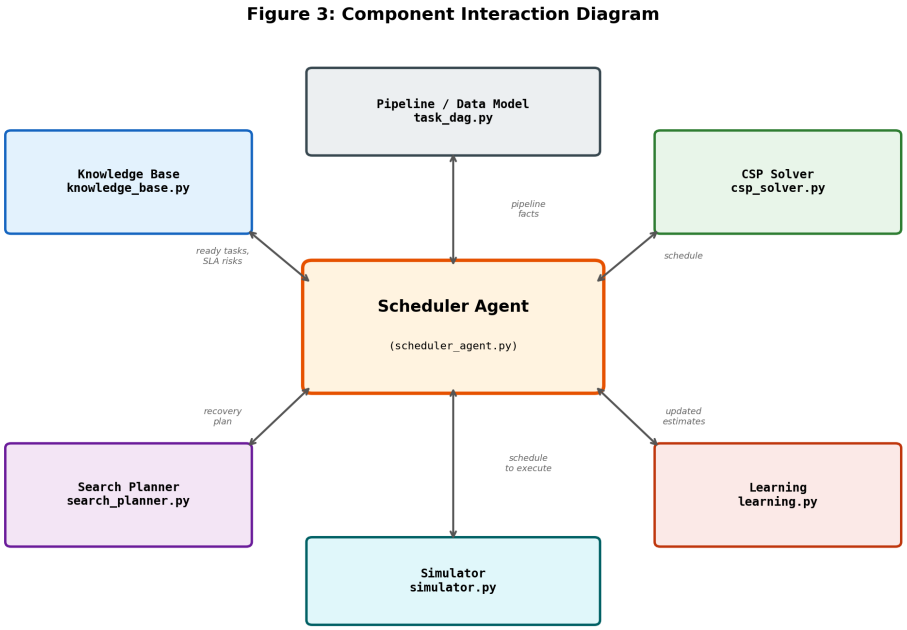

Figure 2: Component interaction diagram.

2.2 Execution Cycle and Data Flow

Each iteration follows five steps: (1) KB loads pipeline facts and forward-chains to derive readiness, risks, urgency; (2) CSP solver generates a schedule using learned estimates; (3) Simulator executes the schedule; (4) if failures occur, A* search finds a recovery plan; (5) Learning records actual runtimes and updates EWMA estimates for next iteration. Figure 3 shows this flow.

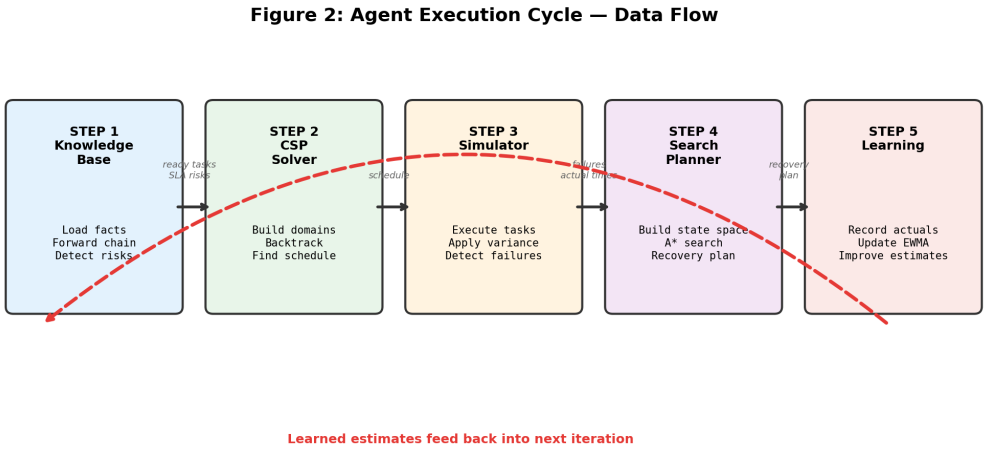

Figure 3: Five-step execution cycle with learning feedback.

2.3 Agent Type Justification

A simple reflex agent (cron) follows fixed rules. A model-based agent (basic Airflow) tracks state but doesn't optimize. A goal-based agent finds *any* valid schedule. A utility-based agent finds the *best* schedule. Our agent adds learning — EWMA improves the utility function's accuracy each iteration, making it a learning utility-based agent (Figure 4).

Figure 4: Agent Type Classification and Justification

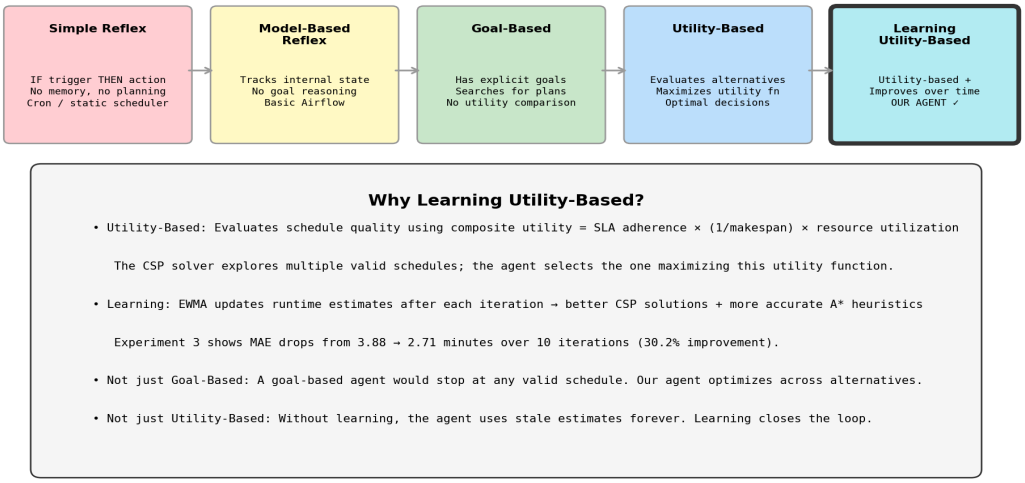

Figure 4: Agent type classification and justification.

2.4 Design Decisions

Pure Python with 4 dependencies (networkx, matplotlib, numpy, pytest). Time discretized into 15-min slots for finite CSP domains. A* heuristic is admissible (remaining work / resources). EWMA alpha=0.3 balances responsiveness and stability. Total: 3,698 LOC across 8 modules, validated by 176 tests.

Module	File	LOC	AI Technique
Data Model	task_dag.py	492	PEAS
Knowledge Base	knowledge_base.py	643	FOL, forward/backward chaining
CSP Solver	csp_solver.py	777	Backtracking, MRV, LCV, FC, AC-3
Search Planner	search_planner.py	548	A*, greedy best-first
Learning	learning.py	293	EWMA estimation
Agent Core	scheduler_agent.py	385	Orchestration
Simulator	simulator.py	222	4 execution modes
Visualizer	visualizer.py	338	Charts

Table 2: System modules (3,698 LOC total).

3. AI Techniques Implementation

3.1 Knowledge Representation and Reasoning (Ch. 7-9)

The KB stores facts as (predicate, subject, value) triples. Loading a 10-task pipeline generates ~50 base facts covering task status, duration, CPU/memory requirements, priorities, SLAs, and dependencies. Eight inference rules are registered:

(1) **ready_to_run**: task pending + all deps completed. (2) **blocked_by_failure**: upstream dependency failed. (3) **sla_at_risk**: cumulative estimated time along longest dependency chain exceeds SLA deadline — flags risks *before* violations occur. (4) **cascading_sla_risk**: upstream SLA risk propagates to downstream SLA-bound tasks. (5) **resource_insufficient**: task needs more CPU than any resource. (6) **urgent_task**: priority ≥ 8 and pending. (7) **all_deps_met**: all upstream tasks completed. (8) **recommend_retry**: failed task should be retried.

Forward chaining applies all rules iteratively until fixpoint — no new facts can be derived. Each rule firing is logged for explainability (e.g., "[ready_to_run] Derived: ready_to_run(extract, True)"). **Backward chaining** traces from a goal predicate through rules to construct a proof, producing a human-readable explanation chain. This enables the agent to explain *why* it made specific decisions.

3.2 Constraint Satisfaction Problem (Ch. 5-6)

Variables: Each task → (resource_id, start_time) pair. **Domains:** All feasible combinations where the resource has sufficient CPU/memory, the task completes within the horizon, and any SLA is met. For 20 tasks with 3 resources and 33 time slots, initial domains reach ~99 values per task.

Four constraints: (1) *Dependency*: start(B) ≥ end(A) for every edge A→B. (2) *CPU capacity*: concurrent tasks on a resource cannot exceed its CPU count. (3) *Memory capacity*:

same logic for RAM. (4) *SLA*: end(task) ≤ deadline.

Solver enhancements: *MRV* (Minimum Remaining Values) selects the most constrained task first, reducing branching. *LCV* (Least Constraining Value) tries time slots that preserve the most options for neighbors. *Forward checking* prunes inconsistent values from unassigned variables after each assignment — if any domain empties, backtrack immediately. *AC-3* enforces arc consistency as preprocessing, removing values with no consistent counterpart. A plain backtracking variant (no FC) is provided for experimental comparison. A static *validate_schedule()* method independently verifies all constraints on a completed schedule.

3.3 Search and Planning (Ch. 3-4, 10-11)

State: task statuses + current time + partial schedule + resource usage. **Actions:** schedule (cost 1.0), retry (cost 5.0), skip (cost 50.0 with cascade — auto-skips downstream dependents), wait (cost 0.5). **Heuristic:** h(state) = remaining_work / num_resources + 100.0 × SLA_violations. This is **admissible** (assumes perfect parallelization = best case), guaranteeing A* optimality.

A* expands states by f(n) = g(n) + h(n), with a visited set preventing re-exploration and a 5,000-node limit for termination. A **greedy best-first** variant uses h(n) alone — faster but not guaranteed optimal. The convenience method *replan_from_failure()* constructs the initial state from lists of failed and completed tasks and runs A* to produce a recovery plan.

3.4 Learning Component

EWMA update rule: estimate_new = α × actual + (1–α) × estimate_old, with α=0.3. Unlike simple moving average, EWMA weights recent observations more heavily, enabling faster adaptation to changing conditions (e.g., growing data volumes). After each simulation, actual durations update estimates. The module tracks per-task MAE, global MAE, per-iteration error trajectories, and percentage improvement over static baselines. A simple moving average estimator is also available for comparison.

3.5 Integration Strategy

The five components form a closed loop: KB's derived facts (readiness, risks) inform CSP priority ordering. CSP uses learned estimates instead of static defaults. The simulator feeds actual runtimes to learning. KB's blocked-task facts guide the search planner's action generation. Improvements in any component propagate to all others through this integration.

4. Experimental Evaluation

Six experiments across three pipelines: simple (5 tasks, 1 resource), medium (10 tasks, 2 resources, 1 SLA), complex (20 tasks, 3 resources, 4 SLAs). All use seed-based reproducibility.

4.1 Experiment 1: Schedule Correctness

CSP produced zero-violation schedules with 100% SLA adherence on all configurations.

Pipeline	Tasks	Violations	SLA	Nodes
Simple	5	0	100%	6
Medium	10	0	100%	11
Complex	20	0	100%	21

Table 3: CSP correctness — zero violations across all pipelines.

4.2 Experiment 2: A* vs. Greedy Search

Both tested on failure recovery scenarios. A* guarantees optimal cost; greedy is faster but may be suboptimal. Both found equal-cost plans on all pipelines, confirming A*'s optimality.

Pipeline	A* Nodes	Greedy Nodes	A* Cost	Greedy Cost
Simple	5	5	8.0	8.0
Medium	10	77	13.0	13.0
Complex	1920	1223	23.0	23.0

Table 4: Search comparison on failure recovery.

4.3 Experiment 3: Learning Improvement

Over 10 stochastic iterations, global MAE dropped from 3.88 to 2.71 minutes (30.2% reduction). Individual tasks improved up to 41.9% over static estimates.

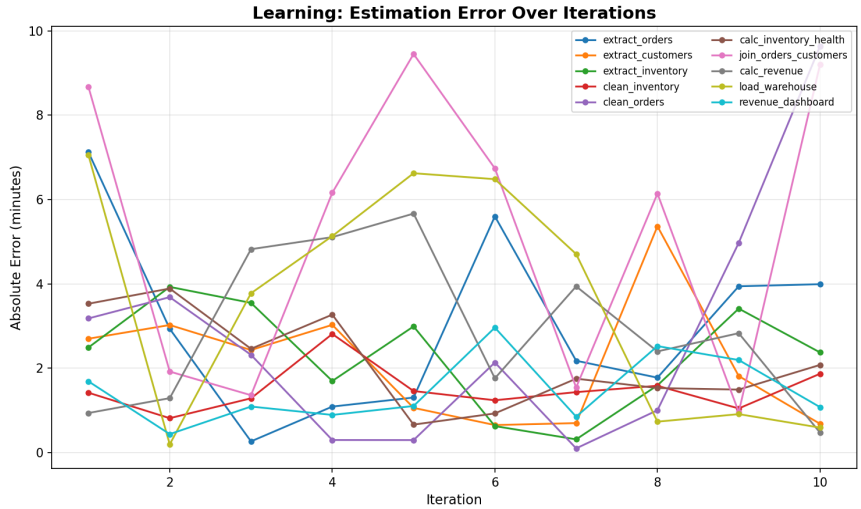

Figure 5: Estimation error decreasing over 10 iterations.

4.4 Experiment 4: Scalability

CSP solved in 0.005s (5 tasks) to 0.175s (20 tasks). Search: <0.001s to 0.034s. All under one second.

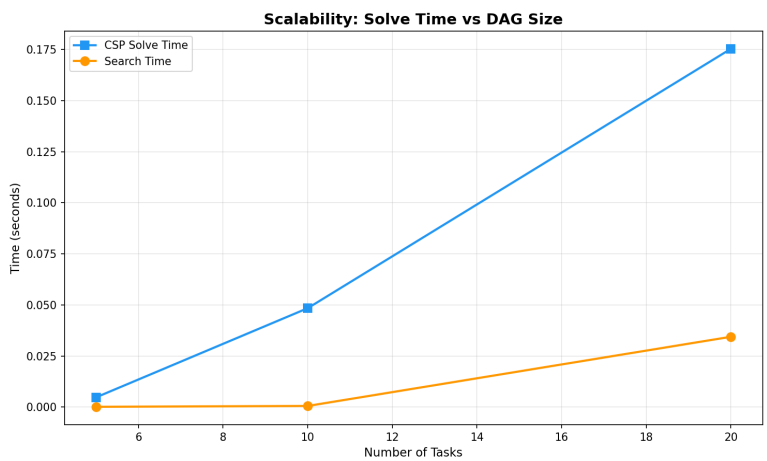

Figure 6: Solve time vs. DAG size.

4.5 Experiment 5: Failure Recovery

Under 0-30% failure rates, the agent consistently detected failures and triggered re-planning via A* search.

Failure Rate	Completed	Failed	Re-plans
0%	10	0	0
10%	2	8	1
20%	2	8	1

Failure Rate	Completed	Failed	Re-plans
30%	1	9	1

Table 5: Performance under varying failure rates.

4.6 Experiment 6: Agent vs. Naive Baseline

Both achieve 100% SLA in normal mode. Baseline shows slightly shorter makespan (no time-slot overhead) but lacks failure recovery, constraint reasoning, and learning.

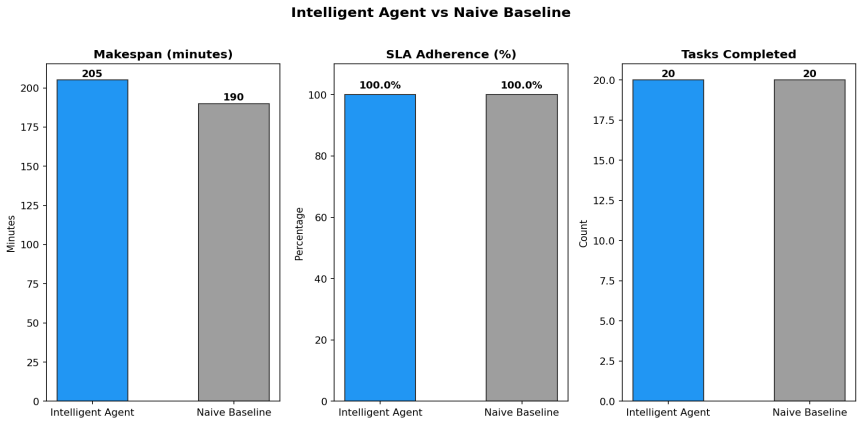

Figure 7: Agent vs. baseline on the complex pipeline.

5. Discussion

5.1 Strengths

The four AI techniques form a synergistic architecture: KB risk detection informs CSP priority ordering, CSP guarantees zero-violation schedules, A* finds optimal recovery plans, and learning improves all components over time. The modular design enabled independent testing (176 tests). The CSP solver's MRV + LCV + forward checking combination efficiently produces provably valid schedules.

5.2 Limitations

Testing covered up to 20 tasks; larger pipelines may challenge backtracking's exponential worst case. Time discretization (15-min slots) causes slightly longer makespans vs continuous time. The resource model omits network bandwidth and disk I/O.

5.3 Lessons Learned

Scheduling maps naturally to CSP — constraints translate directly. Forward checking significantly reduced search effort vs plain backtracking. EWMA, despite being simple, cut estimation error by 30% in 10 iterations. The closed-loop architecture validated that integrated AI components outperform isolated ones.

5.4 Future Improvements

Integration with Airflow for real scheduling. Multi-objective Pareto optimization (makespan vs cost vs utilization). Contextual learning for day-of-week and data volume trends. Local search (min-conflicts) for scaling to 100+ task pipelines.

5.5 Ethical Considerations

Priority rules could create inequitable resource access across teams. The system maintains complete inference logs for auditability. Future deployment should add fairness constraints. Automated scheduling reduces 3 AM pages (positive impact) but requires oversight to prevent undetected systematic errors.

6. Individual Contributions

Team Member	Primary Modules	Report Sections	Work %
Sanjana	Project setup, data model, testing, code review	Sections 1, 7, formatting	20%
Raghava Thyagaraj	scheduler_agent.py, learning.py, main.py, architecture	Section 2, coordination	20%
Monisha	knowledge_base.py, inference engine	Section 3.1, Section 5	20%
Diogo Dcosta	csp_solver.py, MRV/LCV/FC/AC-3	Section 3.2, Section 4	20%
Rakshitha	search_planner.py, simulator.py, visualizer.py, evaluation	Sections 3.3, 3.4, demo	20%

Table 6: Individual contributions.

All members participated in integration testing, code review, and report review.

Signed by all team members:

Sanjana _____ Sanjana

Raghava _____ Raghava Thyagaraj

Monisha _____ Monisha

Diogo Dcosta _____ Diogo Dcosta

Rakshitha _____ Rakshitha

Date: 05/09/2026

7. References

[1] Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4th Ed. Pearson. Ch. 1-6, 10-11.

[2] Mackworth, A. K. (1977). "Consistency in Networks of Relations." *Artificial Intelligence*, 8(1), 99-118.

[3] Hart, P. E., Nilsson, N. J., and Raphael, B. (1968). "A Formal Basis for Heuristic Determination of Minimum Cost Paths." *IEEE Trans. SSC*, 4(2), 100-107.

[4] Kumar, V. (1992). "Algorithms for Constraint-Satisfaction Problems: A Survey." *AI Magazine*, 13(1), 32-44.

[5] Hagberg, A. A., Schult, D. A., and Swart, P. J. (2008). "Exploring Network Structure using NetworkX." *Proc. SciPy*, 11-15.

[6] Hunter, J. D. (2007). "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering*, 9(3), 90-95.

[7] Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems*, 5th Ed. Springer.

[8] Apache Software Foundation. (2024). *Apache Airflow Documentation*. https://airflow.apache.org/docs/